

Learn Low-Level Design in 25 Days: Day 13— Designing a Logging Framework

The Real-World Spark

TechPreneur

Published October 25, 2025

Free: No

Read on Freedium · original

Contents

The Real-World Spark	0
What We're Building	0
The Problem: Why Logging Frameworks Exist	0
Designing the Logging Framework	0
1. Logger	0
2. LogLevel (Enum)	0
3. Appenders	0
4. LoggerFactory	0
Class Diagram (Simplified)	0
Real-World Analogy	0
Try It Yourself	0
Key Takeaways	0
Tomorrow: Building a Rate Limiter	0

The Real-World Spark

You're debugging a production issue at 2 AM. The app crashed, users are angry, and all you have is a single console print: `"Something went wrong."`

No timestamps. No log levels. No clue what went wrong.

That's when every engineer realizes the silent hero in any system isn't the API gateway, the database, or even the caching layer — it's the **logging framework**.



Welcome to **Day 13 of "Learn Low-Level Design in 25 Days"**, where we roll up our sleeves and design a logging framework from scratch.

By the end of this lesson, you'll understand how systems like **Spring Boot, Log4j, and SLF4J** structure their logging layers, how to decouple loggers from outputs, and how to make your own extensible, configurable logging library.

What We're Building

We'll design a **modular Logging Framework** — something that can:

- Log messages at multiple levels (INFO, ERROR, DEBUG, etc.)
- Write to multiple outputs (console, file, or both)
- Be easily extended (e.g., support for remote logging or alerts)

The Problem: Why Logging Frameworks Exist

Let's say you start small:

```
System.out.println("User created successfully!");
System.out.println("Error: Unable to save order.");
```

This works fine… until it doesn't.

In a real-world app:

- You need to **filter logs** (e.g., show only ERRORs).
- You may want to **write logs to a file**, not just the console.
- You want **timestamps, thread info, or custom formats**.
- You might need to **plug in new destinations** — like sending error logs to Slack.

You quickly realize: simple print statements are not enough.

Designing the Logging Framework

Let's think in objects.

We want something like this:

```
Logger logger = LoggerFactory.getLogger("OrderService");
logger.info("Order placed successfully");
logger.error("Payment failed for orderId: 123");
```

Now, let's break down the components.

1. Logger

This is the main class developers interact with. It should:

- Accept log messages
- Delegate the actual writing to "appenders"
- Filter logs based on level

```
public class Logger {
    private String name;
    private LogLevel level;
    private List<Appender> appenders;

    public Logger(String name, LogLevel level) {
        this.name = name;
        this.level = level;
        this.appenders = new ArrayList<>();
    }

    public void log(LogLevel messageLevel, String message) {
        if (messageLevel.ordinal() >= level.ordinal()) {
            for (Appender appender : appenders) {
                appender.append(format(messageLevel, message));
            }
        }
    }

    private String format(LogLevel level, String message) {
        return "[" + LocalDateTime.now() + "] [" + name + "] [" + level + "] " + message;
    }

    public void addAppender(Appender appender) {
        appenders.add(appender);
    }
}
```

2. LogLevel (Enum)

Defines levels like INFO, DEBUG, ERROR.

```
public enum LogLevel {
    DEBUG, INFO, WARN, ERROR
}
```

3. Appenders

These decide where the log goes — console, file, network, etc.

```
public interface Appender {
    void append(String message);
}
```

ConsoleAppender:

```
public class ConsoleAppender implements Appender {
    public void append(String message) {
        System.out.println(message);
    }
}
```

FileAppender:

```
public class FileAppender implements Appender {
    private FileWriter fileWriter;

    public FileAppender(String filename) throws IOException {
        fileWriter = new FileWriter(filename, true);
    }

    public void append(String message) {
        try {
            fileWriter.write(message + "\n");
            fileWriter.flush();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

4. LoggerFactory

Central place to create or reuse `Logger` instances.

```
public class LoggerFactory {
    private static Map<String, Logger> loggerMap = new HashMap<>();

    public static Logger getLogger(String name) {
        return loggerMap.computeIfAbsent(name, n -> {
            Logger logger = new Logger(n, LogLevel.INFO);
            logger.addAppender(new ConsoleAppender());
            return logger;
        });
    }
}
```

Class Diagram (Simplified)

```
LoggerFactory
   |
   +--> Logger
           |
           +--> [Appender] ---> ConsoleAppender
           |               ---> FileAppender
           |
           +--> LogLevel
```

Real-World Analogy

Think of this as your home's security system:

- **Logger** = The control panel
- **Appender** = Where alerts go (speaker, phone notification, police call)
- **LogLevel** = Alert priority (silent alarm vs loud siren)

You can easily plug in a new appender (say, "send log to Datadog") without touching the logger itself. That's the **Open/Closed Principle (OCP)** in action.

Try It Yourself

1. **Add a new appender**: Implement an `EmailAppender` that sends ERROR logs via email (simulate sending for now).
2. **Add configuration**: Can you make the log level configurable at runtime using a `config.json` file?
3. **Challenge**: What would you change if logs were being generated from multiple threads concurrently? (Hint: thread-safe appenders.)

Key Takeaways

- A good logging system is **modular, extensible, and configurable**.
- Separate what is logged (Logger) from where it goes (Appender).
- Follow **SOLID principles** — especially OCP and SRP.
- Add filtering and formatting logic cleanly to avoid bloat.
- Logging frameworks are often your **first debugging friend** in production.

Tomorrow: Building a Rate Limiter

Tomorrow, we'll explore **Rate Limiter Design** — covering **Token Bucket and Leaky Bucket algorithms** — and see how they're used by APIs like Stripe, Netflix, and GitHub to protect their systems from overload.

If you're following the series, save this link: Learn Low-Level Design in 25 Days — Full Series (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)